MVP Development Tasks — Control Plane + Single-Tenant Phase 1 (As-Is)

Scope: build point 1 of the suggested build sequence from `architecture-design.md` — a single-tenant, single-legacy-system slice that proves the full loop: **evidence → agent-extracted ArchiMate model → traceable git commit → human PR review → viewable model.** No multi-tenancy, no Phase 2/3/4, no UML/C4 projection yet — those come after this MVP is working end-to-end.

Written for a team of fresh-grad developers: every task is scoped to be independently completable, with explicit prerequisites so the sequence is unambiguous, and Definitions of Done that are checkable without judgment calls.

---

0. Concrete technology choices for this MVP

The full architecture deliberately left several things pluggable (git host, LLM provider, observability, job queue) so they can vary per client later. For the MVP, pick **one concrete option for each** so the team isn't blocked by open decisions. These are swap-out points later, not permanent commitments — each task below notes where the swap seam is.

Concern

MVP choice

Why

Swap seam for later

Backend language/framework

Python 3.11+, FastAPI

Same language as the Deep Agents runtime — no context-switch between agent code and API code

N/A

Database

PostgreSQL, SQLAlchemy ORM, Alembic migrations

One DB serves both app data and the LangGraph checkpointer

Schema-per-tenant later just adds a `tenant_id` -scoped connection resolver

Agent framework

`deepagents` Python package (LangGraph/LangChain underneath)

This is the whole point of the project

—

LLM provider

Anthropic Claude API (direct)

Simplest to wire up first; Deep Agents' `model=` parameter accepts any LangChain chat model

Swap to a self-hosted model later by changing one constructor argument, not the agent code

Git hosting

GitHub (one repo, acting as "the tenant's model repo")

Fastest to stand up, PR review UI included for free, PAT-based auth is simple

On-prem git adapter (Gitea) is a second implementation of the same internal interface built in Epic G

Job execution

FastAPI `BackgroundTasks` + a `jobs` table for status

Zero extra infrastructure to stand up first

Swap to Celery/RQ/Temporal later behind the same "enqueue job" function signature

Observability

LangSmith (cloud, free tier)

Three environment variables, no code — immediate trace visibility while everything else is being built

Self-hosted Langfuse/OTel later, same env-var-driven wiring pattern

Frontend

React + Vite + TypeScript, plain CSS

Small, standard, fresh grads have likely touched this stack

—

---

1. Quality attribute legend

Referenced by short name in each task below, so it isn't redefined 30 times.

- **Traceability** — can a reviewer always answer "where did this fact come from, and who/what produced it"? Non-negotiable for this project — the whole platform's value is an auditable model, not just a correct-looking one.
- **Security** — credential handling (secrets never in code or git history), input validation, no path traversal, respecting boundaries even before multi-tenancy exists (build the habit now, since Epic B's schema will grow a `tenant_id` later).
- **Reliability / Idempotency** — safe to retry a failed step without corrupting state or double-creating things (a duplicated commit, a duplicate PR, a job stuck "in progress" forever).
- **Reproducibility** — given the same evidence input, the deterministic parts of the pipeline (schema validation, git operations, dedup logic) must behave identically every time. The LLM steps won't be perfectly deterministic — that's expected — but everything around them should be.
- **Maintainability** — clear naming, small functions, comments only where the *why* isn't obvious from the code, so the next developer (possibly a different fresh grad) can extend it without archaeology.
- **Observability** — enough logging/tracing that a failure can be diagnosed from logs/traces alone, without needing to reproduce it live.
- **Usability** — for frontend tasks: can a non-technical reviewer (a client SME) actually use this without a manual?

---

2. Suggested sequencing

Epic A: Environment & Tooling

Epic B: Data Layer

Epic C: ArchiMate Knowledge Base

Epic D: Deep Agent Core Scaffold

Epic E: Ingestion Subagents x5

Epic F: Reconciler + Validator

Epic G: Git Versioning & PR Automation

Epic H: Orchestration & Backend API

Epic I: Frontend Model Viewer

Epic J: End-to-End Validation

Epics A, C, and G can start in parallel on day one (different people, no shared dependency). Epic E's five subagents can be built in parallel by different developers once Epic D is done — they share a schema (Task E0) but not code.

Epic A — Environment & Tooling Setup

A1. Repository & Python environment scaffolding

Description: Create the project repository (this is the *engineering* repo — code — distinct from the *model* repo created in A3, which holds the ArchiMate output). Set up a Python 3.11+ project with a virtual environment, dependency management (`pyproject.toml + uv` or `poetry`), and a basic folder structure separating `backend/` , `agents/` , `frontend/` . Add a `.gitignore` covering `.env` , `__pycache__` , `node_modules` , and virtual environment folders. Add a pre-commit hook or CI check for linting (`ruff`) and formatting (`black`).

Prerequisites: None — this is the first task.

Quality attributes:

- *Maintainability* — folder structure and dependency pinning matter now because five more developers are about to add code into this same repo; an unclear structure compounds fast.
- *Security* — `.gitignore` must be correct *before* anyone commits a `.env` file with real API keys; verify by attempting to add a dummy `.env` and confirming `git status` doesn't see it.

Definition of Done:

- Repo exists with `backend/`, `agents/`, `frontend/` folders and a root `README.md` stub.
- `pip install -e .` (or `uv sync`) succeeds from a clean checkout.
- Linting/formatting run via a single command (e.g. `make lint`) and pass on an empty/starter codebase.
- A test `.env` file is confirmed git-ignored.

A2. PostgreSQL database setup (local/dev)

Description: Stand up a local PostgreSQL instance (Docker Compose recommended — one `docker-compose.yml` at repo root any developer can run with `docker compose up`). Create the database and a dedicated app user (not the Postgres superuser). Add connection configuration via environment variables (`DATABASE_URL`), never hardcoded.

Prerequisites: A1.

Quality attributes:

- *Security* — dedicated non-superuser DB role with only the privileges the app needs (not `SUPERUSER`); connection string via env var, documented in a `.env.example` file with placeholder values (never real credentials committed).
- *Reliability* — `docker-compose.yml` must be reproducible: a teammate running it fresh gets the same schema-ready state, not a manual multi-step setup they have to remember.

Definition of Done:

- `docker compose up` brings up Postgres and the app can connect using `DATABASE_URL` from `.env`.

- `.env.example` exists with all required variables documented (no real secrets).
- A teammate who has never set this up can go from `git clone` to a working DB connection using only the README.

A3. GitHub model repository setup

Description: Create the GitHub repository that will hold the ArchiMate model output for the (single, MVP) legacy system — this is "the tenant's model repo" from the architecture design, scoped down to one tenant/one system for now. Set up the directory layout from the architecture doc:

```
systems/<system-id>/as-is/{motivation,strategy,business,application,technology}/
```

Create a GitHub Personal Access Token (fine-grained, scoped only to this repo, contents + pull-requests read/write) for the backend to use, and store it as an environment variable (`GITHUB_TOKEN`), never in code.

Prerequisites: A1.

Quality attributes:

- *Security* — fine-grained PAT scoped to exactly this one repo, not an org-wide token; if this token leaks, blast radius is one repo, not everything the developer's GitHub account can touch.
- *Traceability* — the directory layout must be created exactly as specified now, because every later task (ingestion subagents, viewer) assumes this exact path structure.

Definition of Done:

- Repo exists on GitHub with the layer folders created (a `.gitkeep` file in each empty folder is fine).
- A fine-grained PAT is generated, scoped to this repo only, and works for a test `git push` from the command line.
- `GITHUB_TOKEN` and `GITHUB_MODEL_REPO` (org/repo name) are documented in `.env.example`.

A4. LangSmith tracing setup

Description: Create a LangSmith account/project (free tier is enough for MVP) and wire up the three standard environment variables (`LANGCHAIN_TRACING_V2` , `LANGCHAIN_API_KEY` , `LANGCHAIN_PROJECT`). Confirm a trivial LangChain call produces a visible trace in the LangSmith UI before any real agent code exists — this is deliberately done early so every subsequent task's agent runs are debuggable from day one.

Prerequisites: A1.

Quality attributes:

- *Observability* — this task exists specifically to make every later task's debugging easier; skipping it "for now" means the team spends Epic E and F debugging agent behavior blind.

Definition of Done:

- A one-line test script making a single LLM call produces a trace visible in the LangSmith project UI.
- Env vars documented in `.env.example` .

Epic B — Data Layer

B1. Core database schema & migrations

Description: Using SQLAlchemy models and Alembic migrations, create the MVP subset of tables from the architecture design (single-tenant, so no `tenant_id` columns yet, but name tables so adding one later is a non-breaking migration):

- `legacy_systems` (id, name, description)
- `model_element_index` (id, system_id, layer, archimate_type, name, git_path, current_commit, updated_at)
- `artifact_versions` (id, system_id, commit_sha, phase, tag, author_type, run_id, approval_status, approved_by, approved_at, created_at)
- `jobs` (id, system_id, phase, status, run_id, error_message, started_at, finished_at)
- `evidence_sources` (id, system_id, source_type, location, description, added_at)

Prerequisites: A2.

Quality attributes:

- *Maintainability* — every table needs a migration, not a hand-run `CREATE TABLE`; this is the habit that keeps schema changes reviewable and reversible for the rest of the project's life.
- *Reproducibility* — running `alembic upgrade head` on a fresh DB must produce an identical schema every time; verify by dropping and recreating the dev DB and re-running migrations.

Definition of Done:

- All five tables exist via Alembic migration (not manual SQL).
- `alembic downgrade -1` / `upgrade head` both work cleanly.
- A schema diagram (can be auto-generated) is committed to `docs/` for the rest of the team to reference.

B2. Data access layer (repository functions)

Description: Write plain Python functions (not raw SQL scattered through the codebase) for the CRUD operations the rest of the system needs: create/get legacy system, upsert model element index rows, create/update artifact version, create/update job status, list evidence sources. Keep these in `backend/repository/` so agent code and API code both call the same functions rather than touching SQLAlchemy sessions directly.

Prerequisites: B1.

Quality attributes:

- *Maintainability* — a single choke point for DB access means a future schema change (e.g. adding `tenant_id`) touches one file, not every place that happened to run a query.
- *Reliability* — job status updates in particular must be safe to call twice with the same result (e.g. marking a job "failed" twice shouldn't error) — this matters once retries show up in Epic H.

Definition of Done:

- Every table from B1 has corresponding create/read/update functions with unit tests using a test database (or a transaction-rollback test fixture).
- Calling an update function twice with the same arguments doesn't raise an error or corrupt data.

Epic C — ArchiMate Knowledge Base

C1. Author the ArchiMate metamodel skill (`archimate-metamodel/SKILL.md`)

Description: This is the single most important non-code artifact in the MVP — it's what stops the agents from inventing invalid ArchiMate elements or relationships. It needs to cover, for the Motivation, Strategy, Business, Application, and Technology layers: the valid element types in each layer (with a one-line definition of each), and a relationship-validity matrix (which of ArchiMate's relationship types — composition, aggregation, assignment, realization, serving, access, influence, triggering, flow, specialization, association — are legal between which element types/layers).

Important note for the team: this is a domain-knowledge task, not primarily an engineering one. A fresh-grad developer's job here is to structure the official ArchiMate 3.2 specification (Open Group) into a clear, lookup-table-friendly `SKILL.md` , **not** to invent or infer the metamodel from general knowledge. Have this reviewed by whoever on the team has ArchiMate/EA expertise before treating it as final — an error here silently propagates into every model the agents ever produce.

Prerequisites: A1 (just needs the repo to exist).

Quality attributes:

- *Traceability* — every rule in this skill should be checkable against the actual Open Group ArchiMate 3.2 specification; don't paraphrase from memory.
- *Maintainability* — structure as tables (element type → layer → definition; relationship type → allowed source/target type pairs), not prose, so the `validator` subagent (Epic F) and future developers can scan it quickly.

Definition of Done:

- `SKILL.md` covers all 5 layers with element type tables and a complete relationship-validity matrix.
- Reviewed and signed off by someone with ArchiMate expertise (flag explicitly if no one on the team has this — in that case, this task blocks on getting that review from the client/consultant side).
- Placed at the path the `SkillsMiddleware` expects to load it from.

C2. Skill smoke-test agent

Description: Write a small throwaway script that creates a minimal Deep Agent with only the `archimate-metamodel` skill loaded, and asks it a handful of test questions ("Is a 'Realization' relationship valid from a Business Process to a Technology Node?", "What layer does 'Application Interface' belong to?"). This isn't a permanent part of the system — it exists purely to catch skill-authoring mistakes (malformed SKILL.md, ambiguous wording) before five other subagents are built depending on it.

Prerequisites: C1, D2 (needs a working Deep Agent instantiation to load skills into).

Quality attributes:

- *Reproducibility* — run the same test questions twice; answers should be consistent on the factual (non-judgment) questions, since these are lookups against a fixed reference, not open-ended reasoning.

Definition of Done:

- At least 8 test questions covering all 5 layers and at least 4 relationship types, with answers manually verified against the spec.
- Any wording ambiguity found is fixed in C1's `SKILL.md` before Epic E begins.

Epic D — Deep Agent Core Scaffold

D0. Common model-element & evidence-citation schema

Description: Define the shared JSON schema every ingestion subagent (Epic E) must produce elements in, and every downstream consumer (reconciler, validator, git writer) reads. At minimum:

```
{
  "id": "string (stable, human-readable slug)",
  "layer": "motivation|strategy|business|application|technology",
  "archimate_type": "string (must match a type in the ArchiMate skill)",
  "name": "string",
  "documentation": "string",
  "confidence": "observed|inferred",
  "evidence": [{"source_type": "string", "locator": "string (file path, doc name+sectio
  "relationships": [{"target_id": "string", "type": "string (must match a relationship
```

```
}
```

Implement this as a Pydantic model shared across all agent code, so malformed output fails loudly (a validation error) instead of silently propagating a broken element into the model.

Prerequisites: C1 (needs to know the valid `archimate_type` and relationship type vocab), A1.

Quality attributes:

- *Traceability* — the `evidence` field is mandatory and non-empty for every element; this is enforced by the schema, not left to agent discipline.
- *Maintainability* — one shared Pydantic model, imported everywhere, so this schema only needs to change in one place.

Definition of Done:

- Pydantic model exists in a shared module (`agents/schema.py`), importable by all subagent code.
- Unit test confirms an element with empty `evidence` fails validation.
- Unit test confirms an `archimate_type` not present in the skill's type list fails validation (cross-check against a parsed version of the skill, or a small hardcoded type list extracted from it).

D1. Filesystem backend configuration

Description: Configure the Deep Agent's file access using `CompositeBackend` : route `/evidence/` to a real, read-only local directory (where sample source code, IaC files, docs, and interview transcripts live for the agent to read — this is *not* git-tracked as part of the model output) and route `/systems/` to a working checkout of the GitHub model repo from A3 (this *is* what eventually gets committed). This is the boundary that keeps "things the agent read" separate from "things the agent produced."

Prerequisites: A3, D2 (needs the base agent to attach the backend to).

Quality attributes:

- *Security* — the `/evidence/` route must be read-only from the agent's perspective (the agent should never be able to modify source evidence); verify by attempting a write and confirming it's rejected.
- *Traceability* — this separation is what makes "evidence vs. produced model" unambiguous later when reviewing a PR diff.

Definition of Done:

- Agent can `read_file` / `glob` / `grep` under `/evidence/` but a `write_file` attempt there fails.
- Agent can `write_file` under `/systems/<system-id>/as-is/...` and the file appears in the local git checkout's working tree.

D2. Base Deep Agent instantiation

Description: Write the code that calls `create_deep_agent()` with: the Claude model configured via the Anthropic API key (env var, never hardcoded), the filesystem backend from D1, and `TodoListMiddleware` (default). Don't wire subagents yet — this task just proves the base agent runs and responds to a trivial prompt, with a trace showing up in LangSmith.

Prerequisites: A4, D1 (can be stubbed initially and revisited).

Quality attributes:

- *Security* — Anthropic API key via env var only; confirm it's in `.env.example` as a placeholder.
- *Observability* — confirm this agent's runs show up in LangSmith before moving on; this is the last checkpoint before subagent complexity starts stacking on top.

Definition of Done:

- `create_deep_agent(...)` instantiates without error and responds to a "hello" style prompt.
- The run is visible as a trace in the LangSmith project from A4.

D3. Placeholder subagent wiring smoke test

Description: Register five placeholder subagents (names matching Epic E's real ones: `strategy-analyst`, `business-analyst`, `code-analyzer`, `infra-analyzer`, `integration-mapper`) via `SubAgentMiddleware`, each with a trivial system prompt ("respond with the literal text 'stub-ok'"). Have the orchestrator's `task` tool call each one in sequence. This proves the delegation wiring works before real prompts/logic are written, so any bugs in Epic E are known to be in the *subagent logic*, not the *wiring*.

Prerequisites: D2.

Quality attributes:

- *Maintainability* — isolating "does delegation work" from "is the subagent's logic correct" saves significant debugging time across the 5 parallel tasks in Epic E.

Definition of Done:

- Orchestrator successfully calls all 5 placeholder subagents via the `task` tool and receives each stub response.
- Trace in LangSmith shows the parent run with 5 nested child runs.

Epic E — Ingestion Subagents

Each of these follows the same shape: read from a defined evidence source type under `/evidence/`, load the `archimate-metamodel` skill, and produce elements conforming to the D0 schema, written to the correct layer path under `/systems/<system-id>/as-is/`. Build these in parallel once D0–D3 are done; they don't depend on each other.

E1. `strategy-analyst` subagent

Description: Reads strategic plans, policy/compliance documents, and business case docs from `/evidence/strategy/` and `/evidence/motivation/` (create these as the intake folder convention). Produces Motivation-layer elements (Stakeholder, Driver, Assessment, Goal, Outcome, Principle, Requirement, Constraint) and Strategy-layer elements (Resource, Capability, Course of Action, Value Stream). Writes output as one JSON file per element under `systems/<system-id>/as-is/motivation/` or `.../strategy/` as appropriate.

Prerequisites: D0, D1, D3, C1.

Quality attributes:

- *Traceability* — every element must cite the specific document/section it came from; reject (log and skip, don't silently drop) any candidate element the agent can't ground in a specific evidence excerpt.
- *Reproducibility* — running against the same evidence twice should produce substantially the same element set (exact wording may vary; the *set of facts* claimed shouldn't).

Definition of Done:

- Given a sample evidence folder (built in Epic J), produces at least one valid element per layer it's responsible for, each schema-valid (D0) and citing real evidence.
- A run against the same sample twice produces the same element count ±1 (documented tolerance for LLM variance).

E2. `business-analyst` subagent

Description: Reads docs/wikis and (for MVP) pre-supplied interview transcripts from `/evidence/business/`. Produces Business-layer elements (Actor, Role, Process, Function, Service). *(Note: live SME interviewing is explicitly out of scope for this subagent — per the architecture design, interviews are conducted by a human via the orchestrator's own thread, not by a stateless subagent; for MVP, assume transcripts are already collected and placed in the evidence folder.)*

Prerequisites: D0, D1, D3, C1.

Quality attributes: Same as E1 (*Traceability*, *Reproducibility*) — apply identically.

Definition of Done: Same shape as E1, scoped to Business-layer elements.

E3. `code-analyzer` subagent

Description: Reads a source code repository and DB schema files from `/evidence/code/`. Produces Application-layer elements (Application Component, Application Service, Data Object, Application Interface). This one can lean more on deterministic signals than the others (e.g. a `grep` / `glob` pass to find service entry points, config files, schema definitions) before handing findings to the LLM for classification into ArchiMate types — encourage the team to use the `grep` / `glob` tools already provided by the filesystem backend rather than dumping entire files into the prompt.

Prerequisites: D0, D1, D3, C1.

Quality attributes:

- *Traceability* — evidence locator should be a real file path + line number/range, not a vague "found in the codebase."

- *Observability* — log which files were actually read per run (helps debug "why didn't it find X" later).

Definition of Done: Same shape as E1, scoped to Application-layer elements, with evidence locators including file path + line range.

E4. `infra-analyzer` subagent

Description: Reads IaC files (Terraform/Ansible) and CMDB-style exports from `/evidence/infra/`. Produces Technology & Infrastructure layer elements (Node, Device, System Software, Technology Service, Artifact).

Prerequisites: D0, D1, D3, C1.

Quality attributes: Same as E3.

Definition of Done: Same shape as E1, scoped to Technology layer elements.

E5. `integration-mapper` subagent

Description: Reads API specs and any available integration documentation from `/evidence/integration/`. Produces cross-layer relationships (serving, flow, realization) connecting elements that the *other four* subagents will have already produced. **This subagent must run last**, after E1–E4 have written their output, since it needs to reference their element IDs.

Prerequisites: D0, D1, D3, C1, and — functionally — the outputs of E1–E4 must exist before this one is meaningfully testable (it can be *built* in parallel, just not *validated end-to-end* until the others produce real output).

Quality attributes:

- *Traceability* — every relationship must reference real element IDs that exist in the current run's output; a relationship pointing to a non-existent ID is a bug, not a valid "inferred" element.

Definition of Done:

- Given E1–E4's sample output, produces relationships referencing valid element IDs, each schema-valid and evidence-cited.
- A relationship referencing an ID that doesn't exist in the current model causes a clear validation error (not a silently written broken file).

Epic F — Model Assembly Subagents

F1. `reconciler` subagent

Description: After E1–E5 write their output files, this subagent (or, for MVP, this can be plain deterministic Python code — see note below) merges near-duplicate elements discovered by different subagents under different names (e.g. two subagents both find "Payment Service" vs. "PaymentSvc"). **MVP simplification, stated explicitly:** implement this as a deterministic normalized-name match (lowercase, strip whitespace/punctuation, compare) rather than the fuzzy-match-plus-LLM-judgment approach described in the full architecture — that upgrade is post-MVP. Flag any near-miss the deterministic pass can't confidently resolve as a `conflict` for human review rather than guessing.

Prerequisites: E1–E5.

Quality attributes:

- *Reproducibility* — this must be deterministic code, not an LLM call, for the MVP version — same input files, same merge decisions, every time.
- *Traceability* — a merged element must retain evidence citations from *all* the elements that were merged into it, not just the first one found.

Definition of Done:

- Given a sample set with at least one intentional near-duplicate (built in Epic J), produces a single merged element retaining evidence from both sources.
- An ambiguous case (name similarity below a defined threshold) is flagged, not silently merged or silently dropped.

F2. `validator` subagent

Description: Checks every element and relationship in the reconciled output against the `archimate-metamodel` skill (valid types, valid relationship pairs) and confirms every element has at least one evidence citation. Produces a coverage/validation report: counts of elements per layer, any schema/metamodel violations found, any elements missing evidence (should be impossible given D0's schema enforcement, but check anyway — defense in depth).

Prerequisites: F1, C1, D0.

Quality attributes:

- *Traceability* — the validation report itself should be a committed artifact (not just console output), since it's what a human reviewer reads before approving a PR.
- *Reliability* — this must not silently pass an invalid model; a violation found should fail the pipeline run loudly (Epic H), not just get logged and ignored.

Definition of Done:

- Given a deliberately-broken test input (an element with an invalid `archimate_type`, a relationship of an illegal type between two layers), the validator flags both.
- Produces a written report file committed alongside the model output.

Epic G — Git Versioning & PR Automation

G1. `commit_to_model` tool

Description: A deterministic Python function (explicitly **not** an LLM-authored action) that takes the current state of the local git checkout's working tree (populated by Epics E/F), creates a feature branch (naming convention: `feature/ingest-<system-id>-<run-id>`), stages and commits the changed files with a descriptive commit message including the `run_id`, and pushes the branch to GitHub.

Prerequisites: A3, D1.

Quality attributes:

- *Reliability / Idempotency* — if this function is called twice for the same `run_id` (e.g. a retried job), it must not create two branches or fail ungracefully; check for branch existence first.

- *Security* — uses the fine-grained PAT from A3 via env var; never logs the token value itself, even in error messages.

Definition of Done:

- Given a populated working tree, produces a pushed feature branch on GitHub with the expected commit message format.
- Calling it twice with the same `run_id` either no-ops safely or produces a clear, handled error — not a crash or a duplicate branch.

G2. `open_pull_request` tool

Description: A deterministic function that opens a GitHub PR from the feature branch created in G1 against `main`, with a PR description auto-generated from the `validator`'s report (F2) — element counts, any flagged conflicts, run metadata (run_id, timestamp). Records the resulting PR number and commit SHA into the `artifact_versions` table (B2) with `approval_status = "pending"`.

Prerequisites: G1, F2, B2.

Quality attributes:

- *Traceability* — the PR description is the primary thing a human reviewer reads to decide whether to approve; it must summarize the validator's findings clearly, not just say "automated update."
- *Reliability* — if PR creation fails after the branch was already pushed (G1 succeeded, G2 failed), the job status (Epic H) must reflect a clear partial-failure state, not silently report success.

Definition of Done:

- Given a pushed branch and a validator report, opens a PR with a description containing element counts and any flagged conflicts.
- `artifact_versions` row created with correct `commit_sha`, `pr number` (can be added as a column, or encoded in a metadata field), and `approval_status = "pending"`.

G3. PR-merge webhook receiver

Description: A FastAPI endpoint that receives GitHub's `pull_request` webhook events, verifies the GitHub webhook signature (using the shared secret configured when the webhook is set up), and — on a `closed` event where `merged = true` — updates the corresponding `artifact_versions` row to `approval_status = "approved"` and refreshes `model_element_index` by re-reading the merged files from the model repo's `main` branch.

Prerequisites: G2, B2, A3.

Quality attributes:

- *Security* — **must verify the webhook signature.** An unverified webhook endpoint is a real vulnerability — anything on the internet could POST a fake "merged" event and mark unreviewed content as approved. This is the one task in the MVP where skipping the security step creates an actual exploitable hole, not just a best-practice gap.
- *Reliability* — GitHub may redeliver the same webhook event; handle duplicate delivery so re-processing an already-approved event is a no-op, not an error or a duplicate index refresh.

Definition of Done:

- Endpoint rejects requests with an invalid/missing signature (test with a deliberately wrong signature).
- Merging a test PR triggers the webhook, and `artifact_versions.approval_status` flips to `approved` within the request.
- Sending the same merge event twice doesn't error or double-process.
- `model_element_index` reflects the merged files' content after processing.

Epic H — Orchestration & Backend API

H1. Top-level Phase 1 orchestrator

Description: Wire the full flow into one orchestrator using `TodoListMiddleware` to plan and track: dispatch E1–E4 (can run in parallel via the `task` tool), then E5, then F1, then F2, then call G1/G2. This is the "one button" that runs the entire Phase 1 loop for a given `system_id` and evidence folder.

Prerequisites: E1–E5, F1, F2, G1, G2.

Quality attributes:

- *Observability* — the todo list itself, visible in LangSmith traces, should give a reviewer a clear step-by-step account of what happened in a given run without reading code.
- *Reliability* — if any step fails (e.g. validator finds a hard violation), the orchestrator must stop and report failure clearly rather than proceeding to commit a broken model.

Definition of Done:

- A single function call (`run_as_is_ingestion(system_id, evidence_path)`) executes the full pipeline end-to-end against sample evidence and results in an open GitHub PR.
- A deliberately-broken evidence set (triggering a validator failure) stops the pipeline before G1/G2 run, with a clear error surfaced.

H2. Async job runner

Description: Wrap H1's orchestrator call in a FastAPI `BackgroundTasks` job so the API can return immediately while the (potentially multi-minute) agent run happens asynchronously. Update the `jobs` table (B1/B2) with `status` transitions: `queued → running → succeeded/failed` , and store the LangSmith `run_id` so a developer can jump from a job row straight to its trace.

Prerequisites: H1, B2.

Quality attributes:

- *Reliability* — a job that crashes must update its own status to `failed` with an error message, not leave a `running` row forever (a fresh grad's first instinct is often to forget the `try/except` around the background task — this is exactly where that bites).
- *Observability* — `run_id` stored on the job row is the link back to LangSmith; this is the bespoke "trace-to-artifact" join called out in the architecture design.

Definition of Done:

- Triggering a job returns immediately with a job ID.
- Job row transitions through the expected statuses and ends in `succeeded` or `failed` (never stuck).
- A forced exception inside the orchestrator results in `failed` status with a non-empty `error_message` .

H3. Backend REST API

Description: Expose the minimal set of endpoints the frontend needs:

- `POST /systems/{system_id}/ingest` — trigger a Phase 1 run (kicks off H2)
- `GET /jobs/{job_id}` — job status
- `GET /systems/{system_id}/elements` — list model elements (from `model_element_index`), filterable by layer
- `GET /elements/{element_id}` — element detail, including evidence citations (reads the actual JSON file from the git repo's `main` branch, since full content isn't duplicated into the DB)
- `GET /systems/{system_id}/artifact-versions` — list versions/PRs with approval status

Prerequisites: H2, B2, G3.

Quality attributes:

- *Security* — even though this is single-tenant for MVP, don't accept a `system_id` from an unauthenticated caller with no check at all; at minimum wire up a basic API key or session check now, since retrofitting auth later tends to get skipped under deadline pressure.
- *Maintainability* — use FastAPI's automatic OpenAPI docs (`/docs`) as the source of truth for the frontend team, rather than a hand-maintained API spec doc that drifts out of sync.

Definition of Done:

- All five endpoints implemented and visible in FastAPI's auto-generated `/docs`.
- Each endpoint has at least one automated test (happy path + one error case).
- Calling an endpoint without valid credentials is rejected.

Epic I — Frontend: Basic Model Viewer

I1. Frontend scaffold

Description: Set up a Vite + React + TypeScript project in `frontend/`, with a basic routing structure (e.g. `react-router`) for the four screens in I2–I4, and an API client module wrapping calls to the backend from H3 (base URL via env var, not hardcoded).

Prerequisites: A1.

Quality attributes:

- *Maintainability* — one shared API client module, not `fetch` calls scattered through components — mirrors the backend's B2 principle on the frontend side.

Definition of Done:

- `npm run dev` serves a working app with placeholder pages for all four planned screens.
- API base URL configurable via `.env`, documented in `.env.example`.

I2. Trigger-run & job status screen

Description: A screen with a button to trigger ingestion (`POST /systems/{id}/ingest`) and a live-updating job status view (simple polling of `GET /jobs/{id}` every few seconds is fine for MVP — no need for websockets yet).

Prerequisites: I1, H3.

Quality attributes:

- *Usability* — a non-technical reviewer should be able to tell, without asking a developer, whether a run succeeded, failed, or is still in progress, and see the failure reason if it failed.

Definition of Done:

- Clicking "run" triggers a job and the screen reflects status changes (queued/running/succeeded/failed) without a manual page refresh.
- A failed job displays the `error_message` from the job row.

I3. Model element browser (list + detail)

Description: A screen listing model elements grouped by ArchiMate layer (`GET /systems/{id}/elements`), and a detail view for a selected element (`GET /elements/{id}`) showing its documentation, evidence citations (source type, locator, excerpt), and its relationships. Link each element's evidence citation, where the locator is a file path, to the corresponding file in the GitHub repo (a simple `https://github.com/.../blob/<commit>/<path>` link is enough — no need to render the file inline for MVP).

Prerequisites: I1, H3.

Quality attributes:

- *Usability* — this screen is the entire point of the MVP for a non-developer reviewer; a client SME should be able to open it and trust what they're looking at because every claim visibly links back to its source.
- *Traceability* — don't summarize evidence away — show the actual citation text, not just "evidence found."

Definition of Done:

- Elements list, grouped by layer, matches what's in `model_element_index` for the selected system.
- Detail view shows documentation, all evidence citations with working links to GitHub, and relationships to other elements (clickable to navigate to the related element).

I4. Artifact version / PR status screen

Description: A screen listing `artifact_versions` rows for the system, showing commit SHA, phase, approval status, and a direct link to the GitHub PR (merged or pending).

Prerequisites: I1, H3.

Quality attributes:

- *Traceability* — this is the screen that answers "which version of the model am I looking at, and was it approved, and by the underlying PR review process."

Definition of Done:

- Lists all artifact versions for a system with correct status and a working link to the actual GitHub PR.

Epic J — End-to-End Validation

J1. Synthetic demo evidence set

Description: Assemble a small, deliberately simple fictional legacy system's evidence: a tiny sample code repo (a couple of services), a short Terraform snippet, one or two short docs, one short "interview transcript" text file, and at least one intentional near-duplicate element opportunity (for testing F1) and one intentional invalid reference (for testing F2/E5). This is the fixture the whole team tests against throughout Epics E–J — build it early enough that Epic E's developers aren't blocked waiting for it, but it can be refined as gaps are found.

Prerequisites: None (can start alongside Epic A).

Quality attributes:

- *Reproducibility* — this fixture must be small and stable enough that every developer gets the same test results; check it into the repo under `test-fixtures/`.

Definition of Done:

- Evidence set checked into the repo, covering all five ingestion subagents' source types, with the intentional edge cases documented in a short README explaining what each is meant to test.

J2. End-to-end acceptance test — the MVP's actual finish line

Description: Run the complete flow against J1's evidence set: trigger ingestion via the API (I2) → confirm all five subagents produce output → confirm reconciliation merges the intentional duplicate → confirm the validator's report is correct (including flagging the intentional invalid reference) → confirm a PR opens on GitHub with a sensible description → **manually review and merge the PR** → confirm the webhook fires and `artifact_versions` flips to approved → confirm the model viewer (I3) shows the merged elements with correct evidence links → confirm the version screen (I4) shows the approved version.

Prerequisites: Every prior epic.

Quality attributes:

- *Reliability* — run this twice from a clean DB/repo state; both runs should succeed the same way (allowing for LLM output wording variance, not for structural failures).

Definition of Done — this is also the Definition of Done for the whole MVP:

- [ ] Ingestion triggered via the UI, not a script.
- [ ] All 5 ingestion subagents produce schema-valid, evidence-cited elements.
- [ ] Reconciler merges the intentional duplicate and retains both evidence sources.
- [ ] Validator correctly flags the intentional invalid reference and this halts auto-progression to PR (per H1's Definition of Done).
- [ ] After fixing/removing the intentional invalid case, a full clean run opens a GitHub PR with an accurate, human-readable description.
- [ ] A human reviewer merges the PR through GitHub's normal UI (proving the HITL gate is a real review, not a rubber stamp).
- [ ] The webhook updates `artifact_versions` to `approved` without manual DB intervention.
- [ ] The model viewer shows the merged elements, grouped by layer, each with working evidence links back to GitHub.
- [ ] The version screen shows the approved artifact version linked to the correct PR.
- [ ] The entire run (from trigger to merged, viewable model) is visible as a connected trace in LangSmith.

J3. Developer runbook / README

Description: Write the README a new fresh-grad teammate (or the same team, three months from now) needs to go from `git clone` to a running local instance: environment setup, `.env` values needed (with where to get each one — Anthropic key, GitHub PAT, LangSmith key), how to run the DB, backend, and frontend locally, and how to run J2's acceptance flow manually.

Prerequisites: J2 (documents the process it validates).

Quality attributes:

- *Maintainability* — the real test of this document is having someone who didn't build the system follow it cold and succeed; don't mark this done from the author's own perspective.

Definition of Done:

- A teammate who did not build the ingestion/git/frontend pieces follows the README from a clean checkout and successfully runs J2's flow without needing to ask for undocumented steps.